

DATASHARE

Plateforme sécurisée de partage de fichiers

Documentation technique du projet

Nom : Mathis GAUTHROT

Année : 2026

Sommaire

Documentation technique du projet	1
Sommaire	2
Introduction	3
Architecture de l'application	4
Choix technologiques justifiés	6
Frontend	6
Modèle de données	9
Documentation API	11
Sécurité et gestion des accès	15
Qualité, tests et maintenance	17
Processus d'installation et d'exécution	20
Utilisation de l'IA	24

Introduction

Datashare est une application web de partage de fichiers développée dans le cadre d'un projet MVP (Minimum Viable Product).

L'objectif du projet est de proposer une plateforme simple et sécurisée permettant :

- Création de comptes utilisateurs,
- Envoi de fichiers,
- Gestion des fichiers déposés,
- Partage via des liens de téléchargement temporaires.

Le projet repose sur une architecture moderne séparant :

- Frontend React,
- API REST Laravel,
- Base de données relationnelle,
- Système de stockage de fichiers.

L'ensemble du projet a été développé dans une logique de maintenabilité, de sécurité et de scalabilité progressive.

Architecture de l'application

Vue générale

Datashare repose sur une architecture web moderne séparant clairement le frontend, le backend et le stockage des données.

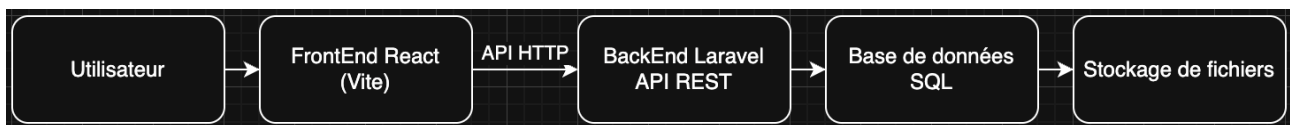
L'application est organisée autour :

- Frontend React chargé de l'interface utilisateur,
- API REST Laravel gérant la logique métier,
- Base de données relationnelle,
- Système de stockage de fichiers.

Cette séparation permet :

- Meilleure maintenabilité,
- Evolution indépendante des différentes couches,
- Réutilisation future de l'API par d'autres clients (mobile, desktop, etc.).

Architecture logique



Frontend

Le frontend est développé avec React et Vite sous forme de Single Page Application.

Il est responsable :

- Authentification utilisateur,
- Interface de gestion des fichiers,
- Envoi des requêtes HTTP vers l'API,
- Expérience utilisateur globale.

L'organisation du frontend repose sur :

- Composants réutilisables,
- Hooks personnalisés,
- Séparation entre logique métier et interface.

Backend

Le backend Laravel expose une API REST sécurisée. Il gère :

- Authentification via Laravel Sanctum,
- Validation des données,
- Gestion des fichiers,
- Autorisations d'accès,
- Interactions avec la base de données.

Les routes protégées utilisent le middleware auth:sanctum.

Stockage des fichiers

Les fichiers envoyés sont stockés sur disque via le système de stockage Laravel.

La base de données conserve uniquement :

- Métadonnées du fichier,
- Propriétaire,
- Token de partage,
- Date d'expiration.

Ce choix réduit la charge de la base de données et simplifie les sauvegardes.

Conteneurisation et environnement de développement

Le projet utilise Laravel Sail et Docker afin d'assurer :

- Environnement de développement reproductible,
- Installation simplifiée,
- Homogénéité entre les postes de développement.

Les services principaux sont conteneurisés :

- application Laravel,
- MySQL,
- Redis,
- Mailpit,
- Meilisearch.

Choix technologiques justifiés

Frontend

React

React a été choisi pour :

- Ecosystème mature,
- Forte adoption dans l'industrie,
- Logique de composants réutilisables,
- Facilité de maintenance.

L'approche composant permet de structurer l'interface en blocs indépendants et réutilisables.

Vite

Vite permet :

- Démarrage rapide du serveur de développement,
- Hot Module Replacement (HMR) performant,
- Temps de build réduits.

Ce choix améliore fortement la productivité durant le développement du MVP.

Tailwind CSS

Tailwind CSS a été retenu afin de :

- Créer rapidement une interface cohérente,
 - Limiter le CSS personnalisé,
 - Accélérer le prototypage de l'interface utilisateur.
-

Backend

Laravel

Laravel a été choisi comme framework backend pour plusieurs raisons :

- Système d'authentification intégré,
- Gestion native des migrations,
- Validation simplifiée des requêtes,
- Système de stockage de fichiers intégré,
- ORM Eloquent,
- Documentation importante,
- Rapidité de développement.

Laravel permet de développer rapidement une API sécurisée tout en conservant une architecture maintenable.

Authentification : Laravel Sanctum

Laravel Sanctum permet de gérer simplement l'authentification API via des tokens Bearer.

Ce choix est adapté à une SPA React et évite la complexité d'une implémentation Auth2 complète pour un MVP.

Sanctum fournit :

- Gestion des tokens,
- Middlewares d'authentification,
- Intégration native avec Laravel.

Base de données relationnelle

Le projet utilise une base de données relationnelle afin de :

- Garantir l'intégrité des données,
- Gérer les relations utilisateur/fichiers,
- Assurer la cohérence applicative.

SQLite est utilisé en développement pour sa simplicité de mise en place.
Une base MySQL peut être utilisée dans un environnement de production.

API REST

L'architecture API REST permet :

- Découplage clair frontend/backend,
- Meilleure maintenabilité,
- Réutilisation potentielle de l'API par d'autres clients.

Les échanges entre le frontend et le backend utilisent des requêtes HTTP et des réponses JSON.

Docker et Laravel Sail

Docker et Laravel Sail permettent de standardiser l'environnement de développement.

Les avantages principaux sont :

- Installation simplifiée,
- Meilleure reproductibilité,
- Réduction des problèmes liés aux différences d'environnement local.

Cette approche réduit les écarts de configuration entre les environnements de développement.

Stockage des fichiers

Les fichiers sont stockés sur disque plutôt qu'en base de données. Cette approche permet de :

- Limiter la taille de la base SQL,
- Améliorer les performances,
- Simplifier les sauvegardes.

Modèle de données

Vue d'ensemble

Le modèle de données de Datashare repose sur une structure relationnelle simple centrée autour de deux entités principales :

- Utilisateurs,
- Fichiers déposés.

Chaque fichier appartient à un utilisateur unique via une relation de type **One-To-Many**.

Cette architecture permet :

- Gestion claire des permissions,
- Bonne intégrité des données,
- Maintenance simplifiée du système.

Entité User

L'entité User représente les comptes utilisateurs de l'application.

Principales informations stockées :

- nom,
- email,
- Mot de passe haché,
- Dates système (created_at, updated_at).

Les utilisateurs sont authentifiés via Laravel Sanctum à l'aide de tokens API.

Entité File

L'entité File représente les fichiers déposés sur la plateforme.

Les métadonnées suivantes sont stockées :

- Nom original du fichier,
- Nom interne généré,
- Chemin de stockage,
- Type MIME,
- Taille du fichier,
- Token de partage,
- Date d'expiration,
- Propriétaire du fichier.

Le contenu physique du fichier n'est pas stocké dans la base de données mais directement sur disque via le système de stockage Laravel.

Relations entre les entités

Le modèle relationnel utilise la relation suivante appelée « One-to-Many » :

- Un utilisateur peut posséder plusieurs fichiers,
- Un fichier appartient à un seul utilisateur.

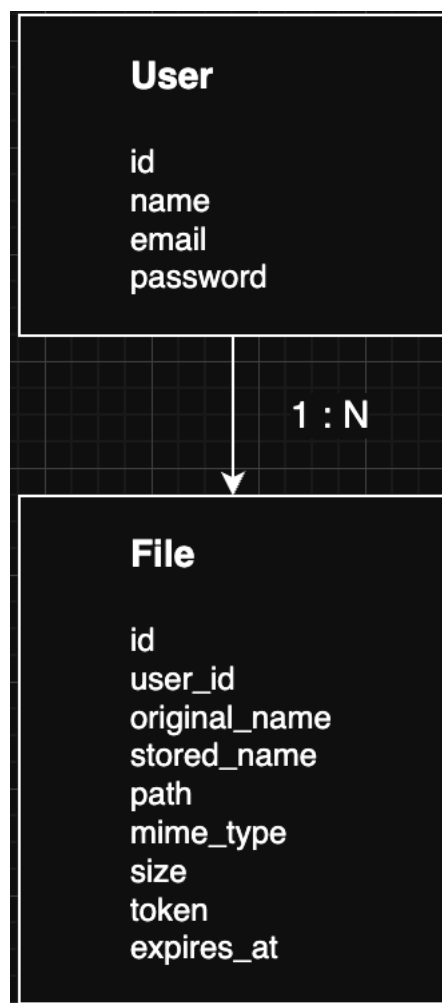
La clé étrangère `user_id` garantit cette relation et applique une suppression en cascade (ON DELETE CASCADE). Ainsi, la suppression d'un utilisateur entraîne automatiquement la suppression de ses fichiers associés.

Contraintes et intégrité des données

Plusieurs mécanismes assurent la cohérence des données :

- Validation des requêtes côté backend,
- Clés étrangères SQL,
- Contraintes de type,
- Politiques Laravel pour les autorisations d'accès,
- Tokens uniques pour le partage des fichiers.

Les dates d'expiration permettent également de limiter l'accès aux fichiers partagés publiquement.



Cette structure relationnelle facilite également les futures évolutions fonctionnelles de l'application.

Documentation API

Présentation

Le backend Datashare expose une API REST sécurisée permettant :

- l'authentification des utilisateurs,
- la gestion des fichiers,
- le partage via liens temporaires.

Les échanges utilisent le format JSON via HTTP.

Les routes protégées utilisent l'authentification Laravel Sanctum via token Bearer.

Base URL

```
http://localhost/api
```

En production, cette URL peut être remplacée par le domaine du serveur.

Authentification

Register

```
POST /register
```

Description : Permet la création d'un compte utilisateur.

Corps de requête :

```
{
  "name": "John Doe",
  "email": "john@example.com",
  "password": "password"
}
```

Réponse succès :

```
{
  "message": "User registered successfully",
  "token": "1|token..."
}
```

Login

POST /login

Description : Authentifie un utilisateur et retourne un token API.

Corps de requête :

```
{  
  "email": "john@example.com",  
  "password": "password"  
}
```

Réponse succès :

```
{  
  "message": "Login successful",  
  "token": "1|token..."  
}
```

Logout

POST /logout

Description : Déconnecte l'utilisateur courant et invalide son token. Le Bearer Token est requis

Réponse succès :

```
{  
  "message": "Logged out successfully"  
}
```

Gestion des fichiers

Upload d'un fichier

POST /files

Description : Permet l'envoi d'un fichier vers le serveur. Le Bearer Token est requis

Données envoyées : multipart/form-data

Paramètres :

Champ	Type	Description
file	File	Fichier à envoyer

Réponse succès :

```
{
  "message": "File uploaded successfully",
  "file": {
    "id": 1,
    "original_name": "document.pdf",
    "token": "uuid-token"
  }
}
```

Liste des fichiers

GET /files

Description : Retourne la liste paginée des fichiers de l'utilisateur connecté. Le Bearer Token est requis.

Paramètres optionnels :

Paramètre	Description
search	Recherche par nom
page	Pagination
sort	Tri des résultats

Affichage d'un fichier

GET /files/{id}

Description : Retourne les informations détaillées d'un fichier. Le Bearer Token est requis.

Modification d'un fichier

PUT /files/{id}

Description : Permet de modifier le nom du fichier et la date d'expiration. Le Bearer Token est requis.

```
{
  "original_name": "new-document.pdf",
  "expires_at": "2026-05-20 12:00:00"
}
```

Suppression d'un fichier

```
DELETE /files/{id}
```

Description : Supprime le fichier physique et les métadonnées associées. Le Bearer Token est requis.

Téléchargement public

```
GET /files/download/{token}
```

Description : Télécharge un fichier via un token public temporaire.

Le backend vérifie :

- l'existence du fichier,
- la validité du token,
- la date d'expiration.

Codes HTTP utilisés :

Code	Signification
200	Succès
201	Ressource créée
401	Non authentifié
403	Accès interdit
404	Ressource introuvable
422	Erreur de validation
500	Erreur serveur

Sécurité des échanges

- Les appels API protégés utilisent : *Authorization: Bearer <token>*
- Les validations des données sont réalisées côté backend via Laravel.
- Les autorisations d'accès aux fichiers utilisent les Politiques Laravel afin de garantir qu'un utilisateur ne peut accéder qu'à ses propres ressources.

Sécurité et gestion des accès

Authentification des utilisateurs

L'authentification de l'application repose sur Laravel Sanctum. Chaque utilisateur authentifié reçoit un token API utilisé pour accéder aux routes protégées.

Les requêtes sécurisées utilisent l'en-tête HTTP suivant :

Authorization: Bearer <token>

Cette approche permet :

- Authentification simple pour une SPA React,
- Sécurisation des routes API,
- Gestion centralisée des accès utilisateurs.

Protection des routes API

Les routes sensibles sont protégées via les middlewares d'authentification Laravel.

Les opérations suivantes nécessitent un token valide :

- Upload de fichiers,
- Consultation des fichiers,
- Modification des métadonnées,
- Suppression des fichiers,
- Déconnexion utilisateur.

Toute requête non authentifiée retourne un code HTTP 401 Unauthorized.

Autorisation et isolation des données

Chaque fichier est associé à un utilisateur unique via la clé étrangère user_id.

Les Politiques Laravel permettent de vérifier qu'un utilisateur :

- Peut uniquement consulter ses propres fichiers,
- Ne peut modifier que ses ressources,
- Ne peut supprimer que les fichiers dont il est propriétaire.

Cette séparation garantit l'isolation des données entre utilisateurs.

Validation des données

Les validations sont réalisées côté backend avant tout traitement.

Les principales vérifications concernent :

- Le format des emails,
- La présence des champs obligatoires,
- La taille des fichiers envoyés,
- Les types MIME autorisés,
- La validité des dates d'expiration.

En cas d'erreur de validation, l'API retourne un code HTTP 422 Unprocessable Entity.

Sécurisation du partage de fichiers

Le partage public des fichiers repose sur des tokens uniques générés automatiquement.

Chaque lien de téléchargement :

- Est associé à un token aléatoire,
- Peut posséder une date d'expiration,
- Vérifie l'existence du fichier avant téléchargement.

Cette approche limite les accès non autorisés aux fichiers partagés.

Protection des mots de passe

Les mots de passe utilisateurs ne sont jamais stockés en clair.

Le backend utilise le système de hachage natif de Laravel afin de sécuriser les informations d'authentification avant leur enregistrement en base de données.

Gestion des erreurs HTTP

L'API utilise les principaux codes HTTP REST standards :

Code	Signification
200	Requête réussie
201	Ressource créée
401	Non authentifié
403	Accès interdit
404	Ressource introuvable
422	Erreur de validation
500	Erreur serveur

Cette standardisation facilite l'intégration frontend/backend et le débogage de l'application.

Qualité, tests et maintenance

Qualité du code

Le projet suit une architecture claire séparant le frontend et le backend afin de faciliter la lisibilité et la maintenance du code.

Le backend Laravel utilise :

- Controllers pour la gestion des requêtes,
- Models Eloquent pour l'accès aux données,
- Requests de validation,
- Policies pour les autorisations,
- Services utilitaires lorsque nécessaire.

Le frontend React est structuré en composants réutilisables permettant :

- Meilleure organisation de l'interface,
- Réutilisation du code,
- Maintenance simplifiée.

Convention et lisibilité

Le code respecte les conventions standards des technologies utilisées :

- Conventions PSR pour PHP,
- Conventions Laravel,
- Composants React modulaires,
- Cassettes utilitaires Tailwind CSS.

Les noms de variables, méthodes et composants sont explicites afin de faciliter la compréhension du projet.

Validation et robustesse

Des validations sont réalisées côté backend afin de garantir l'intégrité des données avant tout traitement.

Les contrôles portent notamment sur :

- Champs obligatoires,
- Formats de données,
- Permissions d'accès,
- Fichiers envoyés,
- Dates d'expiration.

Les erreurs sont retournées sous forme de réponses JSON structurées avec des codes HTTP adaptés.

Gestion des erreurs

Le backend utilise le système de gestion des exceptions Laravel afin de :

- Centraliser les erreurs,
- Retourner des réponses cohérentes,
- Eviter les interruptions critiques de l'application.

Les erreurs API sont standardisées afin de faciliter le traitement côté frontend.

Tests de l'application

Tests backend

Laravel permet l'utilisation :

- Tests unitaires,
- Tests fonctionnels,
- Tests d'API.

Les endpoints peuvent être vérifiés avec :

- Postman,
- Tests PHPUnit,
- Requêtes HTTP automatisées.

Les principaux scénarios testés concernent :

- l'authentification,
- l'upload de fichiers,
- les permissions d'accès,
- le téléchargement public,
- la suppression des ressources.

Tests frontend

Le frontend React peut être vérifié via :

- Tests manuels,
- Tests composants,
- Vérifications responsive.

Les comportements principaux testés sont :

- Authentification utilisateur,
- Affichage des fichiers,
- Formulaires,
- Appels API,
- Gestion des erreurs.

Maintenance et évolutivité

L'architecture choisie facilite les futures évolutions du projet.

L'utilisation d'une API REST découplée permet :

- Ajout d'un client mobile,
- Intégration d'autres interfaces,
- Extension des fonctionnalités backend.

Le système peut évoluer vers :

- Stockage cloud,
- Système de rôles,
- Partage avancé de fichiers,
- Notifications,
- Gestion collaborative.

Environnement de développement

L'utilisation de Docker et Laravel Sail garantit :

- Reproductibilité de l'environnement,
- Installation simplifiée,
- Réduction des problèmes liés aux dépendances locales.

Cette standardisation améliore la stabilité du projet durant le développement et la maintenance.

Processus d'installation et d'exécution

Prérequis

Avant l'installation du projet, les outils suivants doivent être présents sur la machine :

- Docker
- Docker Compose
- Git
- Node.js
- npm

Le projet utilise également :

- Laravel Sail pour l'environnement backend,
- Vite pour le frontend React.

Récupération du projet

Le dépôt peut être cloné avec la commande suivante :

```
git clone https://github.com/mathisGaut/datashare.git
```

Puis :

```
cd datashare
```

Installation du backend

Installation des dépendances PHP

```
composer install
```

Configuration de l'environnement

Copier le fichier .env.example :

```
cp .env.example .env
```

Générer la clé de l'application :

```
php artisan key:generate
```

Démarrage de Docker

Le projet utilise Laravel Sail pour exécuter les services Docker.

Lancer les conteneurs :

```
./vendor/bin/sail up -d
```

Les services suivants sont alors démarrés :

- serveur PHP,
- base de données,
- environnement Laravel,
- services réseau associés.

Migration de la base de données

Créer les tables SQL :

```
./vendor/bin/sail artisan migrate
```

Optionnellement, exécuter les seeders :

```
./vendor/bin/sail artisan db:seed
```

Installation du frontend

Installer les dépendances JavaScript :

```
npm install
```

Lancement du frontend React

Démarrer le serveur Vite :

```
npm run dev
```

Le frontend devient accessible via :

```
http://localhost:5173
```

Lancement de l'API Laravel

Le backend Laravel est accessible via :

```
http://localhost/api
```

Les routes API peuvent être testées avec :

- Postman,
- un navigateur,
- ou des requêtes HTTP.

Stockage des fichiers

Créer le lien symbolique de stockage Laravel :

```
./vendor/bin/sail artisan storage:link
```

Cette commande permet l'accès public aux fichiers stockés dans storage/app/public.

Vérification du fonctionnement

Une fois l'installation terminée, les éléments suivants doivent fonctionner :

- inscription utilisateur,
- authentification,
- upload de fichiers,
- consultation des fichiers,
- téléchargement public,
- suppression des fichiers.

Commandes utiles

Arrêter les conteneurs Docker

```
./vendor/bin/sail down
```

Accéder au terminal Laravel

```
./vendor/bin/sail shell
```

Relancer les conteneurs

```
./vendor/bin/sail up -d
```

Exécuter les migrations

```
./vendor/bin/sail artisan migrate
```

Lancer les tests

```
./vendor/bin/sail artisan test
```

Déploiement

En environnement de production :

- SQLite peut être remplacé par MySQL,
- les variables `.env` doivent être adaptées,
- le mode debug doit être désactivé,
- le serveur web doit pointer vers le dossier `public/`.

L'API peut ensuite être déployée sur un serveur Linux compatible Docker ou PHP/Laravel.

Utilisation de l'IA

Présentation

L'intelligence artificielle a été utilisée comme outil d'assistance durant le développement du projet Datashare.

Son utilisation avait pour objectif :

- d'accélérer certaines phases de développement,
- d'améliorer la productivité,
- de faciliter la rédaction technique,
- et d'assister dans la génération de structures répétitives.

L'IA n'a cependant pas remplacé le travail du développeur : toutes les parties générées ont été relues, adaptées, testées et validées manuellement.

Outils utilisés

Le principal outil utilisé est :

- ChatGPT

L'outil a été utilisé comme assistant de développement pour :

- expliquer certains concepts techniques,
- proposer des exemples de code,
- aider à la correction d'erreurs,
- générer des structures de documentation,
- assister dans la rédaction des endpoints API,
- optimiser certaines requêtes ou composants.

Cas d'utilisation dans le projet

Assistance au développement backend

L'IA a permis :

- d'aider à la mise en place de certaines routes API Laravel,
- de proposer des validations de requêtes,
- d'assister dans l'utilisation de Laravel Sanctum,
- d'aider à structurer certaines migrations et relations Eloquent.

Assistance au développement frontend

Pour la partie React, l'IA a été utilisée afin de :

- générer certains composants de base,
- proposer des structures Tailwind CSS,
- aider à la gestion des états React,
- assister dans l'intégration des appels API.

Assistance à la documentation

L'IA a également été utilisée pour :

- structurer la documentation technique,
- reformuler certaines explications,
- produire des exemples de requêtes API,
- améliorer la clarté rédactionnelle.

Validation humaine

Chaque proposition générée par l'IA a été :

- relue,
- adaptée au contexte du projet,
- testée localement,
- et validée manuellement.

Aucun contenu généré automatiquement n'a été intégré sans vérification préalable.

Avantages observés

L'utilisation de l'IA a permis :

- un gain de temps important,
- une accélération du prototypage,
- une meilleure productivité,
- une assistance rapide lors des blocages techniques,
- et une amélioration de la documentation du projet.

Limites de l'utilisation de l'IA

Certaines limites ont néanmoins été observées :

- propositions parfois inadaptées au contexte exact du projet,
- nécessité de vérifier la sécurité du code généré,
- besoin de corrections manuelles,
- possibilité d'erreurs techniques ou syntaxiques.

L'IA doit donc être considérée comme un outil d'assistance et non comme un remplacement du développeur.

Conclusion

L'intégration de l'intelligence artificielle dans le développement du projet Datashare a permis d'améliorer la rapidité de conception et la productivité globale du développement.

Son utilisation est restée encadrée par une validation humaine systématique afin de garantir :

- la qualité du code,
- la sécurité des fonctionnalités,
- et la cohérence technique de l'application.